PALESTRA · PHP CONFERENCE BRASIL 2026

Não basta funcionar: como tornar aplicações PHP operáveis em produção: E agora?

Observabilidade prática para devs PHP

Jhousyfran Costa

DevOps/SRE · criador do @deploypro.br

Laravel · Symfony · OpenTelemetry

ABERTURA

Quem sou eu

ANTES

Dev PHP e JS

PHP, JavaScript, Laravel e Vuejs. Código meu rodando em produção.

DEPOIS

Infra e DevOps

Kubernetes, Terraform, ArgoCD e GitOps.

HOJE

SRE e observabilidade

Datadog, Grafana, OpenTelemetry e FinOps.

Datadog · CKA · AWS Solutions Architect

ABERTURA

Isso te é familiar?

```
$ tail -f storage/logs/laravel.log
[2026-10-18 03:12:44] production.ERROR: Server Error
[2026-10-18 03:12:51] production.ERROR: Server Error
[2026-10-18 03:13:02] production.ERROR: Server Error
# ...mas qual request? qual usuário? em que serviço?
```

“Funciona na minha máquina.”

“Ontem estava rápido.”

“Deve ser o banco.”

A PROMESSA DE HOJE

Você vai sair sabendo onde está o problema, não só que ele existe.

Conceito · Ferramenta · Demo · Primeiro passo

O PROBLEMA

Monitoring ou observability?

MONITORING

Sabe que quebrou

Pergunta: o sistema está de pé?

Exemplo: alerta de CPU alta ou de taxa de erro 500.

OBSERVABILITY

Sabe por que quebrou

Pergunta: por que esta request ficou lenta?

Exemplo: o trace aponta a query que consumiu o tempo.

Monitoring responde o que você previu. Observability responde o que você ainda não sabia perguntar.

O PROBLEMA

Os três pilares



Logs

O que aconteceu. Eventos com contexto.



Métricas

Quanto e com que frequência. Números ao longo do tempo.



Traces

Por onde a request passou e quanto tempo gastou em cada parada.

Sozinho, cada pilar conta um pedaço. Correlacionados, contam a história inteira.

O PROBLEMA

Por que PHP é difícil de observar

Stateless

Cada request nasce e morre. Nenhum processo guarda contexto entre chamadas.

Filas e workers

O job roda depois, em outro processo, longe da request que o criou.

Muitas dependências

Banco, cache, APIs externas e outros serviços na mesma request.

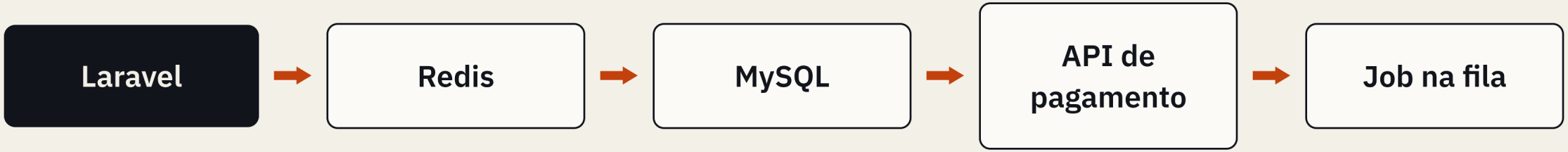
Logs soltos

Cada linha de log, sem nenhum identificador que a ligue à request.

O PROBLEMA

Request lenta. Onde travou?

```
POST /checkout → 200 OK, mas demorou demais
```



Sem trace, cada caixa é um suspeito.

A IDEIA CENTRAL

É aqui que o trace_id salva sua vida.

Um identificador único que acompanha a request por serviços, jobs e logs.

OPENTELEMETRY

O que é OpenTelemetry

Um padrão aberto para gerar e enviar telemetria. Você instrumenta uma vez e escolhe o destino depois.

Padrão aberto

Projeto da CNCF, mantido pela comunidade e por vários fornecedores.

Instrumente uma vez

O mesmo código serve para Grafana, Datadog, Jaeger e outros.

Três sinais, um vocabulário

Traces, métricas e logs falando a mesma língua.

OPENTELEMETRY NO ECOSSISTEMA PHP

“PHP não tem suporte” já era

TRACES	MÉTRICAS	LOGS
Estável	**Estável**	**Estável**

Auto-instrumentação com a extensão nativa ext-opentelemetry, instalada via PECL.

Pacotes prontos no Composer para Laravel, Symfony e outras bibliotecas.

Fonte: opentelemetry.io/docs/languages/php

Laravel e Symfony: o caminho curto

Laravel	`composer require open-telemetry/opentelemetry-auto-laravel`
Symfony	`composer require open-telemetry/opentelemetry-auto-symfony`
Logs (PSR-3)	`composer require open-telemetry/opentelemetry-auto-psr3`

Ganchos automáticos no ciclo de vida do framework, sem mexer no código de negócio.

Pré-requisito: a extensão, com pecl install opentelemetry.

Propagação de contexto

Laravel →	Serviço B	Header traceparent, padrão W3C, nas chamadas HTTP instrumentadas.
Laravel →	Worker	O contexto precisa ir no payload do job. Confira o que a sua versão já faz.
Laravel →	Log	trace_id e span_id em cada linha, via PSR-3, para pular do trace ao log.

A arquitetura em três caixas



O Collector é opcional, mas desacopla a aplicação do destino: trocar de backend vira configuração.

Para onde mandar os dados?

Destino	Bom para	Atenção
Grafana, Tempo e Loki	Stack aberta, com traces, logs e métricas no mesmo lugar.	Você opera e escala a stack.
Datadog	SaaS completo, pouca operação para o seu time.	O custo cresce com o volume de dados.
Jaeger	Começo simples, ótimo para só traces e para a máquina local.	Foco em traces; logs e métricas ficam por sua conta.

Como o dado sai em OTLP, a escolha de hoje não prende a de amanhã.

VAMOS PRA PRÁTICA

Live demo

Laravel, OpenTelemetry e um trace de verdade.

1 App Laravel rodando

2 Uma request lenta

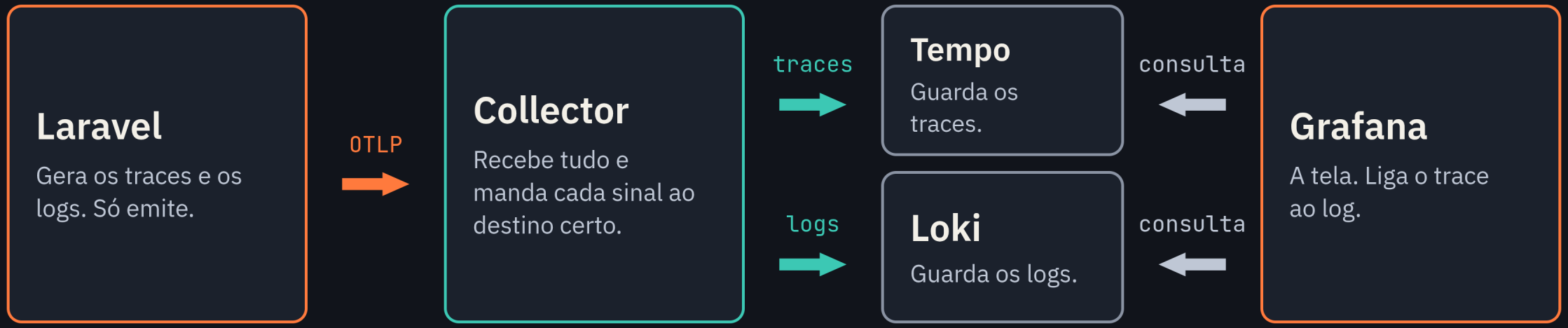
3 Instrumentar com OTel

4 Ver o trace no dashboard

5 Achar o culpado

LIVE DEMO · A STACK

Cinco peças, cada uma com um papel



A aplicação só fala com o Collector. O resto pode mudar sem mexer no código.

LIVE DEMO · PASSO 1

A aplicação em Laravel

```
docker-compose.yml
app:        # Laravel + PHP-FPM
collector: # OTLP na porta 4318
tempo:     # guarda os traces
loki:      # guarda os logs
grafana:   # mostra tudo
```

ENDPOINT

GET /api/pedidos

Lista pedidos com o nome do cliente e a quantidade de itens.

LIVE DEMO · PASSO 2

Uma request lenta

```
public function index()
{
    $pedidos = Pedido::limit(100)->get();

    return $pedidos->map(fn ($p) => [
        'id'      => $p->id,
        'cliente' => $p->cliente->nome,
        'itens'   => $p->itens->count(),
    ]);
}
```

Parece inocente.

Sem erro, sem exceção, nada no laravel.log. Só lentidão.

Onde está o problema?

LIVE DEMO · PASSO 3

Instrumentar com OpenTelemetry

1. Extensão

```
pecl install opentelemetry
```

2. Pacotes

```
composer require open-telemetry/sdk open-telemetry/exporter-otlp open-telemetry/opentelemetry-auto-laravel
```

3. Variáveis de ambiente

```
OTEL_PHP_AUTOLOAD_ENABLED=true
OTEL_SERVICE_NAME=demo-pedidos
OTEL_TRACES_EXPORTER=otlp
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
OTEL_EXPORTER_OTLP_ENDPOINT=http://collector:4318
```

Zero linhas no controller.

LIVE DEMO · PASSOS 4 E 5

O trace aparece



Ilustração do que o Grafana vai mostrar: cada barra é um span, e o comprimento é o tempo.

LIVE DEMO · RESULTADO

O culpado: query N+1

ANTES

201

queries numa única request.

DEPOIS

3

queries, com eager loading.

```
$pedidos = Pedido::with(['cliente', 'itens'])→limit(100)→get();
```

LIVE DEMO · BÔNUS

O log com o mesmo trace_id

.env

```
OTEL_PHP_PSR3_MODE=inject
```

No controller

```
Log::info('Listando pedidos trace_id={trace_id} span_id={span_id}');
```

No laravel.log

```
local.INFO: Listando pedidos trace_id=4bf92f3577b34da6a3ce929d0e0e4736 span_id=00f067aa0ba902b7
```

SINAIS DE SAÚDE

Como saber se a aplicação está bem

Latência

Quanto tempo leva para responder?

`ms · percentis`

Tráfego

Quanta demanda está chegando?

`requests por segundo`

Erros

Que parte das requests falha?

`% de falhas`

Saturação

Quão perto do limite estamos?

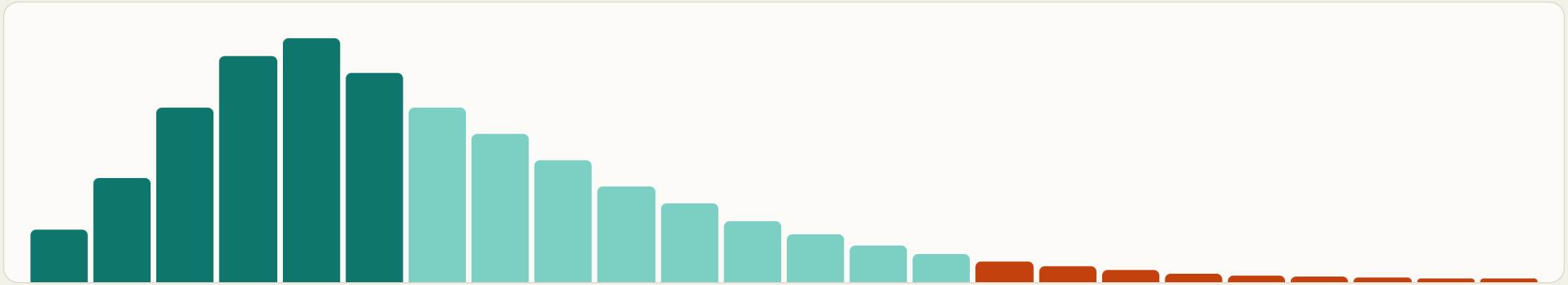
`% de uso`

E o trace? Os quatro sinais dizem *que* algo mudou. O trace diz *onde* e *por quê*.

SINAIS DE SAÚDE · LATÊNCIA

Latência: a média esconde a cauda

Tempo entre receber a request e devolver a resposta. Cada barra é um grupo de requests parecidas.



← mais rápidas

mais lentas →

Metade das requests (até o p50)

Quase todas (até o p95)

Os 5% mais lentos

SINAIS DE SAÚDE · PERCENTIL

O percentil 95 em um exemplo

Exemplo: 20 requests, em milissegundos, da mais rápida para a mais lenta.

80	85	90	90	95	100	100	105	110	110
115	120	120	130	140	150	160	180	**250**	1800

■ p95 ≈ 250 ms: a 19ª de 20 (95% de 20 = 19) ▢ A 20ª ficou de fora: 1800 ms

MÉDIA
≈ 207 ms

MEDIANA (P50)
≈ 112 ms

P95
≈ 250 ms

95 de cada 100 requests foram mais rápidas que o p95.

SINAIS DE SAÚDE · PERCENTIL

Por que olhar o p95 e não só a média

Mostra quem sofre

A média dilui os casos ruins. O p95 mostra a experiência dos 5% mais lentos.

A cauda é comum

Numa página com 20 chamadas, a chance de ao menos uma cair nos 5% lentos é de 64%.

Aponta a causa

Cauda alta costuma ser N+1, lock no banco ou serviço externo lento.

p50
O caso típico

p95
Bom para alertas e SLOs

p99
Casos raros, com mais ruído

SINAIS DE SAÚDE · ERROS

Taxa de erro: quantas requests falham



FÓRMULA

taxa de erro = requests com falha ÷ total de requests

O QUE CONTA

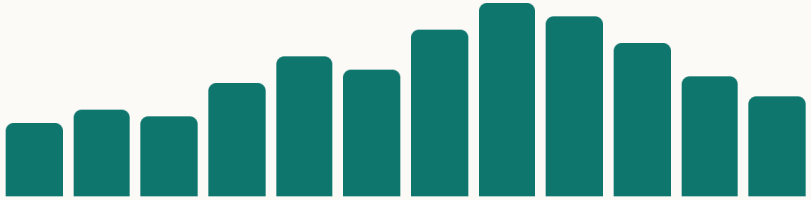
- 5xx e exceções não tratadas
- Timeouts e falhas de dependências
- 4xx só se vierem em massa

SINAIS DE SAÚDE · TRÁFEGO E SATURAÇÃO

Quanta demanda e quanto aguentamos

TRÁFEGO

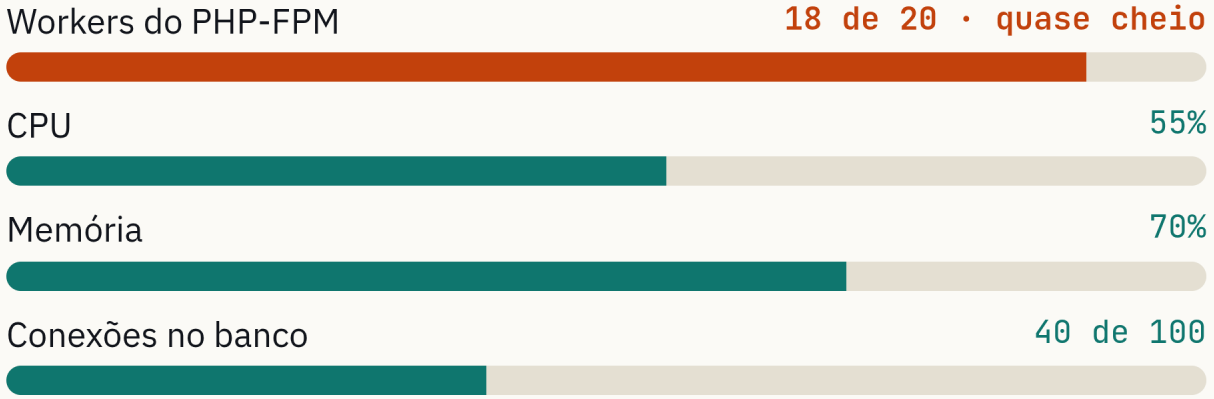
Requests por segundo



A demanda que chega. Sozinho não diz se está bom, mas dá contexto aos outros sinais.

SATURAÇÃO

Quão perto do limite



Também vale olhar: o atraso da fila de jobs e a taxa de acerto do cache e do OPcache.

SINAIS DE SAÚDE · JUNTANDO TUDO

Do sinal até a causa

1 · SINAL

Algo mudou

O p95 do /api/pedidos subiu e o alerta disparou.

2 · TRACE

Onde e por quê

Um trace lento mostra 201 spans de banco dentro da request.

3 · LOG

Os detalhes

O log da mesma request, pelo mesmo trace_id.

Métrica avisa. Trace localiza. Log detalha.

BUSINESS CASE

Tempo para achar a causa

Sem observability

[__ horas]

Alerta → Palpites → Logs → Reproduzir → Causa

Com tracing

[__ min]

Alerta → Abrir o trace → Causa

BUSINESS CASE

Os dois custos

Custo de não observar

- Horas de engenharia gastas em palpite
- Servidor maior para esconder código lento
- Cliente que percebe antes de você

Custo de instrumentar

- Uma extensão e alguns pacotes
- Volume de dados para armazenar
- Amostragem para segurar a conta

Observabilidade também custa. A diferença é que você escolhe quanto, e vê o retorno.

PARA LEVAR

Observabilidade não é custo. É seguro contra horas de war rooms.

ENCERRAMENTO

Comece pequeno: três passos

1

Instale

Extensão, SDK e o pacote do seu framework, em staging.

2

Um dashboard

Latência por endpoint e a lista dos traces mais lentos.

3

Um alerta

Só no endpoint mais crítico: latência alta ou erros.

Não tente observar tudo de uma vez.

ENCERRAMENTO

Para continuar estudando

Documentação oficial	opentelemetry.io/docs/languages/php
SDK e exemplos	github.com/open-telemetry/opentelemetry-php
Instrumentações	github.com/open-telemetry/opentelemetry-php-contrib
Pacotes no Packagist	open-telemetry/opentelemetry-auto-laravel
Código desta demo	Jhousyfran/php-conference-2026-demo

PERGUNTAS?

Obrigado!

De var_dump e reza a trace e certeza.

Jhousyfran Costa

DevOps/SRE · consultoria em DevOps e FinOps

@deploypro.br · /in/jhousyfran · jhousyfrancosta@gmail.com